UNIDAD 1 - RECONOCIMIENTO DE LAS CARACTERÍSTICAS DE LOS LENGUAJES DE MARCAS

- UNIDAD 1 - RECONOCIMIENTO DE LAS CARACTERÍSTICAS DE LOS LENGUAJES DE MARCAS
  - INTRODUCCIÓN
  - TIPOS DE LENGUAJES DE MARCAS
    - LM según su Ámbito de Aplicación
  - EVOLUCIÓN de los Lenguajes de Marcas
    - GML hacia SGML
    - SGML hacia HTML y XML
    - HTML hacia HTML5
      - Evolución de HTML a HTML 5
    - Evolución de XML
      - Tecnologías de apoyo de `XML`
    - JSON (JavaScript Object Notation)
  - TABLA COMPARATIVA `XML`, `HTML`, `JSON`, `SVG` y `MathML`
  - XML - ESTRUCTURA Y SINTAXIS
    - Características Principales
      - Conceptos clave
    - Entidades XML
      - Tabla Ejemplo de Entidades XML
  - .DTD
    - Componentes principales de un DTD
      - `<!ELEMENT>` - Declaración de elementos
      - `<!ATTLIST>` - Lista de atributos
    - Tipos de datos de atributos
      - `ID` - Identificador único
      - `IDREF` - Referencia a un ID
      - Enumeración de valores - `(valor1 | valor2 | ...)`
    - Obligatoriedad de atributos
      - `#REQUIRED` - Atributo obligatorio
      - `#IMPLIED` - Atributo opcional
      - `#FIXED` - Valor fijo
      - Valor por defecto
    - Contenido de elementos
      - `#PCDATA` - Parsed Character Data
      - `EMPTY` - Elemento vacío
      - `ANY` - Cualquier contenido
    - Ejemplo completo de DTD y XML
    - Resumen de conceptos clave

INTRODUCCIÓN

Los **Lenguajes de Marcas** son una herramamienta principal en el tratamiento y la organización de la información. Su **principal función** es aportar estructura y significado a los datos, facilitando su almacenamiento, intercambio y visualización.

No se debe confundir los Lenguajes de Marcas con los Lenguajes de Programación. En los Lenguajes de Marcas, la **información se codifica a través de texto plano e información adicional** (como etiquetas, marcas o anotaciones) definida en un .DTD (Document Type Definition).

De esta manera, en un documento **.XML se almacenan y estructuran a los datos**, y el **.DTD contiene las reglas que garantizan la estructura del .XML** .

Para definir el vínculo de un archivo *.DTD* dentro de un archivo *.XML* se indica: `<!DOCTYPE ... SYSTEM "fichero.dtd">`.

Hoy día el estandar es usar **XSD (XML Schema)** como reemplazo del *.DTD*. La estructura permite definir tipos de datos y restricciones. **El vínculo a un *.XSD* se hace con:** `xsi:noNamespaceSchemaLocation="fichero.xsd"`.

TIPOS DE LENGUAJES DE MARCAS

- **PRESENTACIÓN**: Dan **formato visual** al texto. Ej: HTML.
- **PROCEDIMIENTOS**: No usados hoy en día en ámbito web o empresarial.
- **DESCRIPTIVOS/SEMÁNTICOS**: Describen **la estructura y significado de los datos**. Ej: XML

LM según su Ámbito de Aplicación

> **Documentación electrónica e intercambio de datos**: Para estructurar datos y compartir información de manera estandarizada. Entre ellos:
>
> - **XML**→ Lenguaje base para muchos otros.
> - **JSON**→ Muy usado en aplicaciones web para intercambio de datos.

> **Tecnologías de Internet**: Orientados a web. Permiten estructuras páginas o mostrar contenido. Entre ellos:
>
> - **HTML**→ Estructurar páginas web, junto con *RSS y ATOM* (para la sindicación y actualización de noticias).

> **Lenguajes especalizados**: Son dialectos (basados en XML) diseñados para resolver necesidades concretas. Entre ellos:
>
> - **MathML**→ Para fórmulas matemáticas.
> - **SVG**→ Para gráficos vectoriales.
> - **FXML o XAML**→ Para interfaces gráficas en aplicaciones.

EVOLUCIÓN de los Lenguajes de Marcas

GML hacia SGML

`GML (Generalized Markup Language)` fue creado por *Charles F. Goldfarb*.

> `GML` es el antecesor de `SGML(Standard Generalized Markup Language)`, que deriva en *`XML`* y *`HTML`*.

SGML hacia HTML y XML

`SGML (Standard Generalized Markup Language)` creado por Charles Goldfarb al convertirlo en estándar ISO.

> - Pensado para ser **flexible y potente**, permitiendo definir cualquier tipo de documento con **etiquetas personalizadas**.
> - De `SGML` surgen `HTML` y `XML`, versiones más simples y prácticas.

HTML hacia HTML5

`HTML (HyperText Markup Language)`. Creado en 1989 por Tim Berners-Lee (creador de la Web).

- `HTML` Es un lenguaje que permite compartir y enlazar información entre documentos de distintas fuentes.

> `HTML` Se basa en dos ideas:
>
> - `ASCII` → Estándar de codificación de caracteres.
> - `SGML` → Metalenguaje que permite definir etiquetas para estructurar documentos.

`HTML` permitió describir documentos con enlaces (hipertexto), imágenes, listas, tablas... para ser mostrado en navegadores.

Posteriormente `HTML` ha evolucionado en `HTML 2` → 3 → 4 → `XHTML` → `HTML 5`(Actualidad)

Evolución de HTML a HTML 5

> - `HTML (1991)` → Muy básico: texto, enlaces e imágenes.
> - `HTML 2.0 (1995)` → Primer estándar oficial. Añade formularios y tablas simples.
> - `HTML 3.2 (1997)` → Soporta applets, scripts (JavaScript) y más opciones de presentación.
> - `HTML 4.01 (1999)` → Separa contenido (HTML) de presentación (CSS). Incluye formularios avanzados y accesibilidad.
> - `HTML5 (2014)` → Versión actual. Soporta audio, video, canvas (gráficos), geolocalización, almacenamiento local, etiquetas semánticas (`<header>`, `<footer>`, `<article>`), responsive y multimedia sin plugins.

Evolución de XML

`XML (eXtensible Markup Language)` se crea en 1998 como estándar estructural. Sólo incluye **estructura y datos**.

- **Características de** `XML`

- Es un **metalenguaje** → Sirve para crear otros lenguajes.
- **Permite definir tus propias etiquetas** → Decides los nombres según tu necesidad (`<libro>`, `<autor>`, `<precio>`).
- **Permite asignar atributos a esas etiquetas** → Por ejemplo (`<libro categoría="novela">`)
- Exige un **esquema exacto** → Sintaxis bien formada y validarse con reglas **(DTD/XSD)**
- **Separa estructura de diseño** → `XML` solo organiza los datos. El diseño se aplica después con `XSLT` o `CSS`.

Tecnologías de apoyo de `XML`

- `XSL (eXtensible Stylesheet Language)` → para transformar y dar formato a XML.
- `XML Namespaces` → para evitar conflictos de nombres cuando mezclas distintos XML.
- `XML Schemas (XSD)` → para definir reglas y validar documentos XML.
- `XML Linking Language (XLink, XPointer)` → para crear enlaces avanzados dentro de documentos XML.
- `XPath` → para navegar y buscar dentro de un XML.

JSON (JavaScript Object Notation)

Nació ligado a `JavaScript`, pero a dia de hoy es **independiente y usado en cualquier lenguaje**

Su función principal **almacenar e intercambiar datos entre sistemas**.

Es **más ligero y sencillo que XML**, y utiliza una estructura **clave → valor**, facilitando su lectura.

No tiene tantas herramientas como XML *(sin `.DTD` ni `.XSD`, ni espacios de nombres)*.

- **Características: Basado en dos estructuras**

- **Objetos** → entre `{}` con pares `clave:valor`
- **Arrays** → entre `[]` (`listas ordenadas de valores`).

TABLA COMPARATIVA `XML`, `HTML`, `JSON`, `SVG` y `MathML`

Característica	XML	HTML	JSON	SVG	MathML
TIPO de LM	**Descriptivo/Semántico:** Almacenar, estructurar e intercambiar datos	**Presentación:** Mostrar información en páginas web	**Descriptivo/Semántico:** Almacenar e intercambiar datos	**Descriptivo/Semántico:** Aplicación de XML para mostrar dibujos vectoriales	**Descriptivo/Semántico:** Aplicación de XML para fórmulas matemáticas
Etiquetas / Estructura	Etiquetas creadas por el usuario. Utiliza `pares nombre="valor"`	Etiquetas predefinidas por el estándar. `<p>`, `<div>`, `<img>`	Utiliza una estructura `clave -> valor`. `{}` Para Objetos. `[]` Para Arrays (listas)	Utiliza etiquetas predefinidas por el estándar	Utiliza etiquetas predefinidas por el estándar
Sintaxis	**Estricta:** bien formado, etiquetas cerradas	**Flexible:** Permite etiquetas sin cerrar y atributos sin comillas	**Muy estricto:** Comillas dobles obligatorias, sin comas finales	**Estricto**: *Sigue las reglas de XML*	**Estricto**: *Sigue las reglas de XML*
Ámbito de aplicación	**Documentación electrónica e intercambio de datos:** Define estructura y significado	**Tecnologías de Internet:** Define la presentación de los datos con CSS y JS	**Documentación electrónica e intercambio de datos:** Define estructura y significado	**Lenguaje especializado:** Describe gráficos vectoriales	**Lenguaje especializado:** Describe fórmulas matemáticas

Ejemplo	`<XML>`
	`<libros>`
	`<titulo>Lenguajes de Marcas</titulo>`
	`</libros>`
		`<div class="libros">`
		`<h2>Libros</h2>`
		`<ul>`
		`<li>Lenguajes de Marcas</li>`
		`</ul>`
		`</div>`
			`{`
			`"libros": [`
			`{`
			`"titulo": "Lenguaje de marcas"`
			`}`
			`]`
			`}`
				-	-

XML - ESTRUCTURA Y SINTAXIS

Características Principales

- Orientado a la **organización estructurada de datos**.
- Compatible con protocolos `HTTP` o `URL`.
- Cumple normas `XML`, y en parte, también de `SGML`.
- No requiere grandes conocimientos de programación.
- Fácil de crear y **legible por humanos y máquinas**.
- **Diseño formal, conciso y ordenado**.
- **Extensible y adaptable** (puedes crear tus propias etiquetas).
- Compatible con cualquier procesador XML.

Conceptos clave

- **Marcado** → conjunto de etiquetas que estructuran el documento.
- **Datos carácter** → La información contenida dentro de esas etiquetas.
- **ESTRUCTURA** dividida en **Prólogo** y **Ejemplar**
  - **Prólogo** → Contiene la declaración inicial y otras referencias, como información técnica del documento, indica la versión y el conjunto de carácteres usados.
  - **Ejemplar** → Contenido real con los datos estructurados en etiquetas.

> Los documentos XML pueden incluir comentarios no procesados por el intérprete y se escriben entre `<!--` **Comentario de ejemplo : D** `-->`.

Los **comentarios** pueden colocarse en cualquier parte excepto:

> - Antes del prólogo `<?xml version="1.0"?>`
> - Dentro de una etiqueta.

Entidades XML

Las **entidades** son una forma de representar carácteres especiales que confundirían al intérprete si se escriben directamente, o por no estar disponibles en el teclado.

Se escriben siempre con **Ampersand** `&`. Ejemplos:

Tabla Ejemplo de Entidades XML

Entidad	Representación	Descripción
`<`	<	Menor que
`>`	>	Mayor que
`&`	&	Ampersand
`'`	'	Apóstrofo
`"`	"	Comillas dobles

.DTD

Un **DTD (Document Type Definition)** es un documento que **define la estructura y las reglas de validación** que debe seguir un archivo XML. Actúa como un "manual de instrucciones" que especifica qué elementos pueden existir, qué atributos tienen, si son obligatorios u opcionales, y cómo se relacionan entre sí.

El DTD garantiza que el XML esté **bien formado** (sintaxis correcta) y **válido** (cumple las reglas definidas).

Componentes principales de un DTD

Un DTD está compuesto principalmente por:

- **Declaraciones de elementos** (`<!ELEMENT>`)
- **Declaraciones de atributos** (`<!ATTLIST>`)
- **Entidades** (referencias a caracteres especiales o contenido reutilizable)

`<!ELEMENT>` **- Declaración de elementos**

Define **qué elementos (etiquetas) pueden existir** en el XML y **qué contenido pueden tener**.

Sintaxis:

```
<!ELEMENT nombre_elemento (contenido)>
```

Ejemplos:

```
<!ELEMENT nombre (#PCDATA)>
```

→ El elemento `<nombre>` contiene solo texto.

```
<!ELEMENT proyecto (nombre, descripcion, tareas)>
```

→ El elemento `<proyecto>` debe contener, **en este orden**: `<nombre>`, `<descripcion>` y `<tareas>`.

```
<!ELEMENT proyectos (proyecto+)>
```

→ El elemento `<proyectos>` contiene **uno o más** elementos `<proyecto>`.

Símbolos de cardinalidad:

Símbolo	Significado
`+`	Uno o más
`*`	Cero o más
`?`	Cero o uno (opcional)
`,`	Secuencia (orden específico)
`|`	Alternativa (uno u otro)

`<!ATTLIST>` **- Lista de atributos**

Define **qué atributos puede o debe tener un elemento**, su tipo de dato y si son obligatorios u opcionales.

Sintaxis:

```
<!ATTLIST nombre_elemento
    nombre_atributo tipo_dato obligatoriedad>
```

Ejemplo:

```
<!ATTLIST proyecto
    id ID #REQUIRED
    estado (pendiente | en_progreso | finalizado) #REQUIRED>
```

→ El elemento `<proyecto>` tiene dos atributos:

- `id`: de tipo `ID` (identificador único), **obligatorio**.
- `estado`: solo puede tener los valores `pendiente`, `en_progreso` o `finalizado`, **obligatorio**.

Tipos de datos de atributos

`ID` **- Identificador único**

Define un atributo cuyo valor debe ser **único en todo el documento XML**. Se utiliza para identificar elementos de forma inequívoca.

Ejemplo:

```
<!ATTLIST empleado id ID #REQUIRED>
```

```
<empleado id="emp1">...</empleado>
<empleado id="emp2">...</empleado>
```

→ Cada `id` debe ser único. No puede haber dos elementos con `id="emp1"`.

`IDREF` **- Referencia a un ID**

Define un atributo que **hace referencia a un** `ID` **existente** en el documento. Se utiliza para establecer relaciones entre elementos.

Ejemplo:

```
<!ATTLIST asignadoA refEmpleado IDREF #REQUIRED>
```

```
<empleado id="emp1">
    <nombre>Juan Pérez</nombre>
</empleado>

<tarea id="t1">
    <titulo>Diseñar la base de datos</titulo>
    <asignadoA refEmpleado="emp1"/>
</tarea>
```

→ El valor `"emp1"` en `refEmpleado` debe coincidir con algún `id` definido en el documento.

Enumeración de valores - `(valor1 | valor2 | ...)`

Los **paréntesis con valores separados por** `|` indican que el atributo **solo puede tener uno de esos valores específicos**. Es una lista cerrada de opciones.

Ejemplo:

```
<!ATTLIST tarea prioridad (alta | media | baja) #REQUIRED>
```

```
<tarea prioridad="alta">...</tarea>  ☑ Válido
<tarea prioridad="urgente">...</tarea>  ❌ Error (no está en la lista)
```

Obligatoriedad de atributos

`#REQUIRED` **- Atributo obligatorio**

Indica que el atributo **debe aparecer siempre** en el elemento.

Ejemplo:

```
<!ATTLIST proyecto id ID #REQUIRED>
```

```
<proyecto id="p1">...</proyecto>  ☑ Correcto
<proyecto>...</proyecto>  ❌ Error: falta el atributo id
```

`#IMPLIED` - **Atributo opcional**

Indica que el atributo es **opcional**. Puede estar presente o no en el elemento.

Ejemplo:

```
<!ATTLIST tarea prioridad (alta | media | baja) #IMPLIED>
```

```
<tarea id="t1" prioridad="alta">...</tarea>  ☑ Válido
<tarea id="t2">...</tarea>  ☑ Válido (sin prioridad)
```

`#FIXED` - **Valor fijo**

Indica que el atributo tiene un **valor fijo predefinido** que no puede cambiarse.

Ejemplo:

```
<!ATTLIST documento version CDATA #FIXED "1.0">
```

```
<documento version="1.0">...</documento>  ☑ Válido
<documento version="2.0">...</documento>  ✖ Error
```

Valor por defecto

Se puede definir un **valor predeterminado** que se asignará automáticamente si el atributo no aparece.

Ejemplo:

```
<!ATTLIST tarea prioridad (alta | media | baja) "media">
```

```
<tarea id="t1">...</tarea>  → Automáticamente toma prioridad="media"
<tarea id="t2" prioridad="alta">...</tarea>  → Usa el valor especificado
```

Contenido de elementos

`#PCDATA` - **Parsed Character Data**

Indica que el elemento puede contener **texto plano** que será procesado por el intérprete XML.

Ejemplo:

```
<!ELEMENT nombre (#PCDATA)>
```

```
<nombre>Juan Pérez</nombre>
```

`EMPTY` - **Elemento vacío**

Indica que el elemento **no puede contener ningún contenido**, solo puede tener atributos.

Ejemplo:

```
<!ELEMENT asignadoA EMPTY>
<!ATTLIST asignadoA refEmpleado IDREF #REQUIRED>
```

```
<asignadoA refEmpleado="emp1"/>  ☑ Correcto
<asignadoA refEmpleado="emp1">Texto</asignadoA>  ✖ Error
```

`ANY` **- Cualquier contenido**

Indica que el elemento puede contener **cualquier tipo de contenido** (texto, otros elementos, etc.). Se usa raramente porque no impone restricciones.

Ejemplo:

```
<!ELEMENT nota ANY>
```

Ejemplo completo de DTD y XML

```
<!ELEMENT proyectos (proyecto+)>

<!ELEMENT proyecto (nombre, descripcion, tareas)>
<!ATTLIST proyecto
    id ID #REQUIRED
    estado (pendiente | en_progreso | finalizado) #REQUIRED>

<!ELEMENT nombre (#PCDATA)>
<!ELEMENT descripcion (#PCDATA)>

<!ELEMENT tareas (tarea+)>
<!ELEMENT tarea (titulo, asignadoA)>
<!ATTLIST tarea
    id ID #REQUIRED
    prioridad (alta | media | baja) #IMPLIED>

<!ELEMENT titulo (#PCDATA)>

<!ELEMENT asignadoA EMPTY>
<!ATTLIST asignadoA
    refEmpleado IDREF #REQUIRED>

<!ELEMENT empleados (empleado+)>
<!ELEMENT empleado (nombre, email)>
<!ATTLIST empleado
    id ID #REQUIRED>
```

XML válido según este DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE proyectos SYSTEM "proyectos.dtd">
<proyectos>
    <proyecto  id="P1" estado="pendiente">
        <nombre>Diagramas de flujo 1DAW</nombre>
        <descripcion> Preparar una unidad donde tratemos diagramas de flujo</descripcion>
        <tareas>
            <tarea id="T1" prioridad="alta">
                <titulo>Hacer explicacion y teoria con ejemplos practicos en una presentacion de google</titulo>
                <asignadoA refEmpleado="E001"/>
            </tarea>
```

```
            <tarea id="T2">
                <titulo>Hacer ejercicios y correciones</titulo>
                <asignadoA refEmpleado="E002"/>
            </tarea>
        </tareas>
    </proyecto>
    <empleados>
        <empleado id="E001">
            <nombre>Juan Antonio</nombre>
            <email>juanantonio@gmail.com</email>
        </empleado>
        <empleado  id="E002">
            <nombre>Maria Jesus</nombre>
            <email>mariajesus@gmail.com</email>
        </empleado>
    </empleados>
</proyectos>
```

Resumen de conceptos clave

Término	Significado
`<!ELEMENT>`	Define qué elementos existen y qué contienen
`<!ATTLIST>`	Define los atributos de un elemento
`ID`	Atributo con valor único en el documento
`IDREF`	Atributo que referencia a un `ID` existente
`#PCDATA`	El elemento contiene texto procesable
`EMPTY`	El elemento está vacío (sin contenido)
`#REQUIRED`	El atributo es obligatorio
`#IMPLIED`	El atributo es opcional
`#FIXED`	El atributo tiene un valor fijo
`(valor1|valor2)`	El atributo solo puede tener esos valores (enumeración)
`+`	Uno o más elementos
`*`	Cero o más elementos
`?`	Elemento opcional (cero o uno)